NAmat: MATLAB Neighborhood Algorithm User Manual

Teddie Hall, Maggie Curry
North Carolina State University

May 10, 2026

1 Introduction

NAmat implements the Neighborhood Algorithm (NA), introduced in two papers by Malcolm Sambridge in 1999, in MATLAB for use in geophysical inversion problems. The MATLAB code is based on the neighpy Python implementation developed by Auggie Marignier [1]. The NAmat code is stored in this GitHub repository. Given a model for a geophysical process and data produced by that process, inversion methods can be used to estimate missing parameters in the model. In general, a sample of candidate values for the missing parameters is generated, the forward model is computed for each of these samples, and the theoretical output of the model is compared to the empirical data using an objective function which measures the model's misfit. Then, inference is performed on the samples to find the most plausible candidate parameter values. The NA performs this process in two phases: the search phase and the appraisal phase [2].

The NA uses a geometric structure called the Voronoi diagram (also called a Dirichlet diagram) to accelerate the convergence of the search phase and to approximate the posterior probability distribution (PPD) of the parameters in the appraisal phase. Given n data points within region R, the Voronoi diagram partitions R into n-many cells such that each cell is the nearest-neighbor region of the corresponding data point based on Euclidean distance [2]. Figure 1 shows an example of a Voronoi diagram.

1.1 Search Phase

The search phase randomly generates a set of samples within the search space and computes misfit for each sample. It performs an initial search by randomly generating n_i samples within the entire search space with a uniform distribution and computes the misfit of these samples using the objective function. Then the NA performs the following N-many times: the samples are ranked by misfit, the n_r samples with the lowest misfit are selected to be resampled, n_r random walks are performed from the set of resampled cells using Gibbs sampling to produce n_s new samples [2]. The file `NASearcher.m` defines the class that performs this phase.

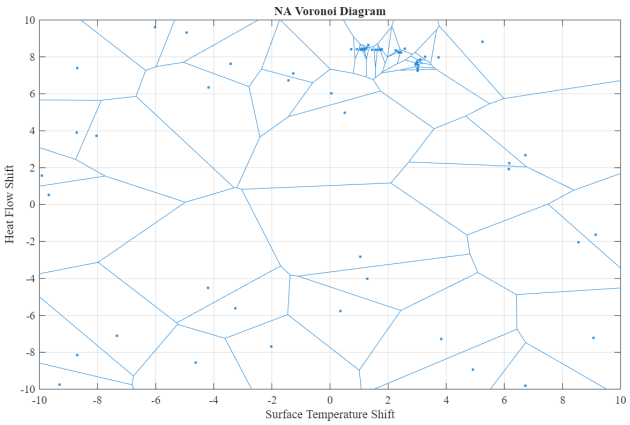


Figure 1: A Voronoi diagram.

1.2 Appraisal Phase

The appraisal phase uses Monte Carlo integration to estimate the joint PPD and performs inference on this density function. It does so by resampling from the samples generated in the search phase and performing random walks through the search space, starting from the resampled points. The negative misfit associated with the Voronoi cell in which the endpoints of each random walk land are used to approximate the log-transformed posterior probability density. The sample mean, sample mean error, covariance, and sample covariance error are computed from the new samples to provide point estimates and uncertainty quantification for each of the parameters of interest [3]. The file `NAAppraiser.m` defines the class that performs this phase. The file `MCIntegrals.m` defines a class that is used throughout the appraisal phase to accumulate samples and estimate the mean, sample mean error, covariance, and sample covariance error.

2 Instructions

2.1 Initialize and Run the Search Phase

1. Define the bounds of the search space with the `bounds` matrix, a $2 \times n_d$ matrix where each row contains the lower and upper bounds of the search space for the n_d parameters of interest.

2. Define the number of iterations (`N`), the number of samples generated per iteration (`ns`), the number of cells to resample (`nr`), and the number of samples in the initial search (`ni`).

3. Define the `objective` function. The objective function function must take single argument of type array and return a float. Any additional data or variables needed for the objective function can be passed into the `NASearcher` object using the `args` argument.

4. (Optional) Select a `seed` for the random number generator to make the results replicable.

5. Initialize the NASearcher object with the following constructor:

```
nas = NASearcher(@objective, ns, nr, ni, N, bounds, args, seed);
```

Note: in Matlab, the objective function must be passed to the NASearcher constructor using an ampersand (@) before the function name.

6. Run the search phase using the run function:

```
par = true; %Setting to run the search phase in parallel
nas.run(par);
```

The run function can be executed in parallel by setting the argument par to true or serial by setting it to false. By default, it executes in parallel if possible.

7. Access the array of samples generated with the property nas.samples and the misfit values for each sample with the property nas.objectives.

2.2 Initialize and Run the Appraisal Phase

1. Define the number of resamples (n_resample) and the number of walkers (n_walkers). Setting n_walkers to 1 will run the code in serial, and greater than 1 will run the code in parallel if possible.
2. Define the save argument to determine whether to save the new samples generated during the appraisal phase.
3. Define the starting fraction (start_fraction) if running in parallel. Otherwise, the default value is 0.5.
4. Initialize the NAAppraiser object with the following constructor:

```
naa = NAAppraiser(n_resample, n_walkers, nas, seed);
```

5. Run the search phase using the run function:

```
naa.run(save, start_fraction);
```

6. Access the mean, sample mean error, covariance, and sample covariance error using the following properties: naa.mean, naa.sample_mean_error, naa.covariance, and naa.sample_covariance_error.

It is also possible to appraise preexisting samples and corresponding misfit values without running the search phase with the following syntax:

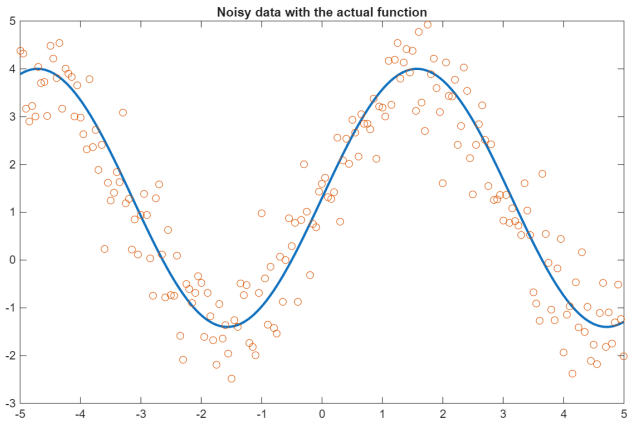
```
% Define initial_ensemble, log_ppd, and bounds matrices
% initial_ensemble: (nd x N) matrix of samples within the search space
% log_ppd: (1 x N) column vector of misfit for each sample
% bounds: (2 x nd) matrix of bounds of the search space
naa = NAAppraiser(n_resample, n_walkers, NA, seed,
                  initial_ensemble, log_ppd, bounds);
naa.run(save, starting_frac);
mean = naa.mean
mean_err = naa.sample_mean_error
```

```
cov = naa.covariance
cov_err = naa.sample_covariance_error
```

3 Example

The following example uses simulated data to demonstrate usage of the NAmat package and potential plots of the NA output. First, simulate noisy data:

```
% Sine Wave Example Function
beta1 = 1.3;
beta2 = 2.7;

x = -5:0.05:5;
y = beta1 + beta2*sin(x);

% Add noise to the line
y_noise = y + 0.7 * randn(size(y));

% Plot
figure;
p = plot(x, y); p.LineWidth = 2; hold on; scatter(x, y_noise);
title('Noisy data with the actual function'); hold off;
```



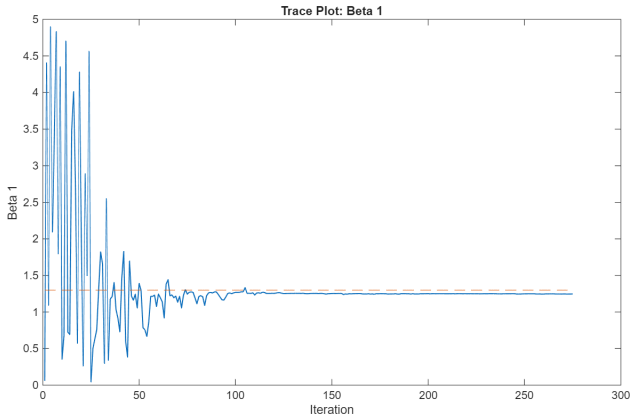
Define the arguments for the search phase, then initialize the NASearcher object and run:

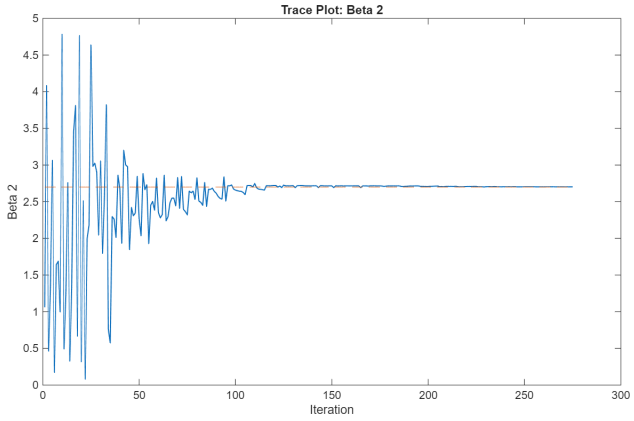
```
%%%% Search Phase %%%%
% Define bounds for the two parameters
bounds = [0 5; 0 5];

% Define struct for additional objective function arguments
args.x = x;
args.y = y_noise;

% Define RMSE as the objective function
function output = obj_fn(in, args)
    pred = in(1) + in(2) * sin(args.x);
    output = rmse(pred, args.y);
end
```

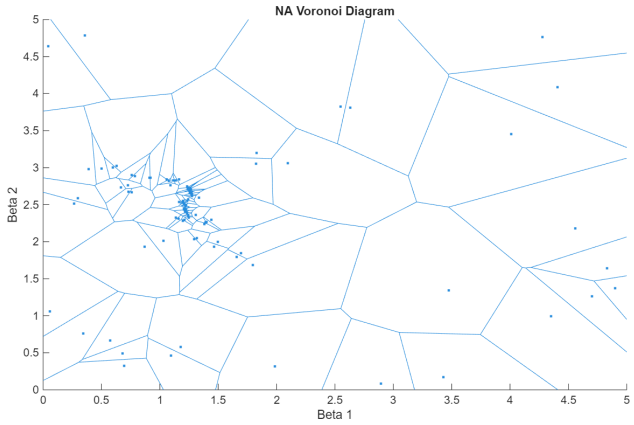
```
ns = 10;     % number of samples generated per iteration
nr = 8;      % number of cells to resample
ni = 25;     % number of samples in initial search
N = 25;      % number of iterations
seed = 42;

% Initialize NASearcher object
nas = NASearcher(@obj_fn, ns, nr, ni, N, bounds, args, seed);
nas.run(false);

% Trace plots of both parameters
trace_x = 1:size(nas.samples, 1);

figure;
p = plot(trace_x, nas.samples(:, 1));
p.LineWidth = 1;
hold on;
plot(trace_x, ones(size(trace_x))*beta1, '--')
xlabel('Iteration'); ylabel('Beta 1');
title('Trace Plot: Beta 1')
hold off;

figure;
p = plot(trace_x, nas.samples(:, 2));
p.LineWidth = 1;
hold on;
plot(trace_x, ones(size(trace_x))*beta2, '--')
xlabel('Iteration'); ylabel('Beta 2');
title('Trace Plot: Beta 2')
hold off;

% Voronoi diagram
figure; hold on;
voronoi(nas.samples(:,1), nas.samples(:,2));
xlabel('Beta 1'); ylabel('Beta 2'); title('NA Voronoi Diagram');
hold off;
```

The trace plots for β_1 and β_2 show how the algorithm explores the entire search space for the initial search (iterations 1 through 25 in this example), then begins to converge toward the most likely value of each parameter by minimizing the misfit of each sample. The trace plots display the true value of β_1 and β_2 with a red dashed line.

The Voronoi diagram shows the joint locations of the samples, (β_1, β_2), and their Voronoi cells. The convergence of the algorithm into areas with lower misfit results in higher densities of points in areas that are more likely to contain the true values of the parameters. The Voronoi cells will have smaller area in these regions of higher likelihood. This surface is used to approximate the log-transformed PPD of new samples generated by the appraisal phase.



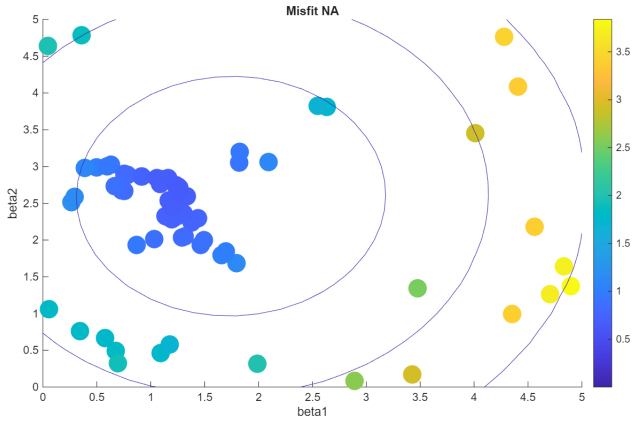




Define the arguments for the appraisal phase, then initialize the NAAppraiser object and run:

```
%%%% Appraisal Phase %%%%
n_resample = 1000;
n_walkers = 1;
starting_frac = 0.5;
save = true;

naa = NAAppraiser(n_resample, n_walkers, nas, seed);
naa.run(save, starting_frac);
mean = naa.mean
```

```
mean_err = naa.sample_mean_error
cov = naa.covariance
cov_err = naa.sample_covariance_error

% Plot estimated MVN distribution
x1 = 0:.2:5;
x2 = 0:.2:5;
[X1,X2] = meshgrid(x1,x2);
X = [X1(:) X2(:)];

y = mvnpdf(X,mean,cov);
y = reshape(y,length(x2),length(x1));

figure;
scatter(nas.samples(:,1), nas.samples(:,2),[size(nas.objectives,1)],
    nas.objectives,'filled');
hold on; colorbar;xlabel('beta1'); ylabel('beta2'); title('Misfit NA');
contour(x1,x2,y,[0.0001 0.001 0.01 0.05 0.15 0.25 0.35]); hold off;
```



The scatter plot above shows the samples from the search phase colored by misfit (RMSE) overlaid with the Multivariate Normal (MVN) distribution of the parameter search space estimated by the appraisal phase. The MVN distribution has a mean of $[1.7464, 2.5964]^T$ and a covariance of $\begin{bmatrix} 1.1210 & 0.0147 \\ 0.0147 & 1.4501 \end{bmatrix}$. The sample error of the mean and covariance is $[1.7464, 2.5964]^T$ and $\begin{bmatrix} 0.0479 & 0.0437 \\ 0.0437 & 0.0514 \end{bmatrix}$, respectively.

4 Developer Notes

The NAmat package was developed using the neighpy Python package as reference (located in this GitHub repository) [1]. Some notable changes were made in the translation from Python to MATLAB:

search.py, line 9 There is no equivalent to a Python `__call__` function in MATLAB, so I did not create an `ObjectiveFunction` class like in the Python code.

search.py, line 80 As opposed to the n_r-many `SeedSequence` objects used in the Python code to generate unique random number streams, the MATLAB code uses a single `RandStream` object with n_r substreams.

search.py, line 111 The `_initial_random_search()` function uses the NumPy function `rng.uniform()` to create a matrix of uniformly distributed random numbers within the bounds defined by the `bounds` property with dimensions $n_i \times n_d$. This was accomplished in MATLAB using the functions `rand()` to generate uniformly distributed random numbers between 0 and 1, then using `repmat()` to create scaling matrices to transform the numbers to be within the appropriate bounds.

appraise.py, line 215 I replicated the behavior of the `argmin()` and `argmax()` NumPy functions for masked arrays in MATLAB by manually replacing the masked values with a fill value of `1e+20` and `-1e+20`, respectively, before computing the argmin or argmax of the matrix. Otherwise, this recursive function gets stuck in an infinite loop.

References

[1] Auggie Marignier. *neighpy*. URL: https://github.com/auggiemarignier/neighpy/tree/main.

[2] Malcolm Sambridge. "Geophysical inversion with a neighbourhood algorithm—I. Searching a parameter space". In: *Geophys. J. Int.* 138 (1999), pp. 479–494.

[3] Malcolm Sambridge. "Geophysical inversion with a neighbourhood algorithm—II. Appraising the ensemble". In: *Geophys. J. Int.* 138 (1999), pp. 727–746.